



**"RANKED 45
IN INDIA**

#University Category



**WORLD
UNIVERSITY
RANKINGS**

**NO.1 PVT. UNIVERSITY IN
ACADEMIC REPUTATION IN INDIA**



**ACCREDITED GRADE 'A'
BY NAAC**



**PERFECT SCORE OF 150/150 AS A TESTAMENT
TO EXCEPTIONAL E-LEARNING METHODS**

1



Applied Machine Learning

Unit 06 – Lecture 17: Neural Networks and Backpropagation

Tofik Ali

School of Computer Science, UPES Dehradun

Autumn 2026

One equation, applied over and over: the chain rule

Topic focus: **backpropagation, derived and checked**

Repository: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 8–9.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 8–9.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 4–5, 7, 14.; Zhang et al., *Dive into Deep Learning*, Ch. 5.

Quick Links

The perceptron

XOR

Activations

Backpropagation

Training an MLP

Initialisation

Recurrent networks

Summary

Agenda

The perceptron: one unit, one rule

XOR: the wall, and the way through

Activation functions, and why they are not optional

Backpropagation, derived in full

An MLP, from scratch, on real data

Where the weights start

Recurrent and feedback architecture

Summary

How this lecture works

Every idea appears twice

1. **By hand** — four rows of a truth table, a network with nine parameters, three time steps. Small enough that every number can be checked with a pen.
2. **In code** — the same arithmetic in NumPy, then the same arithmetic again inside a library, so you can see the library is doing exactly this and nothing magic.

Commit before you look

Five slides ask you to predict a number before it is revealed. Write your guess down. Being wrong on paper is how the idea sticks.

The centrepiece is **backpropagation worked in full on a 2–2–1 network**: the forward arithmetic, then all nine partial derivatives, then a numerical check that every one of them is right. That derivation is the most examinable thing in Unit VI.

Where we are

The last lecture built many small models

An ensemble takes a mediocre learner, makes hundreds of copies that disagree, and averages them. The capacity comes from the **number of models**.

This lecture builds one large model

A neural network is a single function with thousands of parameters, fitted by gradient descent. The capacity comes from the **number of parameters**, and every one of them is adjusted by the same

Today's one question

A model with ten thousand parameters. How do I work out which way to move *each* of them?

And the answer is embarrassingly simple

The chain rule. Applied once per layer, from the loss backwards. One forward pass and one backward pass give you all ten thousand derivatives — not ten thousand passes. That asymmetry is the difference between possible and impossible, and it is the whole of **backpropagation**.

One seed — the convention for this lecture

Why a network needs one

A network is **initialised at random** and its mini-batches are **shuffled at random**. “Run it and see” means nothing without a seed: change the seed and the weights, the losses and the ratios all move.

The one seed

Everything arbitrary in this deck uses **seed 0**: `np.random.default_rng(0)` for every weight draw, `random_state=0` for every scikit-learn estimator.

Two streams from one seed

Where a run needs the weights *and* the shuffle order, the second is an offset — `default_rng(0 + 100)` — not a second invented number.

The seed that is not a seed

`make_moons(random_state=0)` *defines* the two-moons dataset. Change it and you have different data, not a different experiment.

A neuron is a weighted sum and a threshold

$$z = \sum_{i=1}^d w_i x_i + b, \quad \hat{y} = \varphi(z)$$

The three parts

The **weights** w_i say how much each input matters and in which direction. The **bias** b shifts the threshold. The **activation** φ turns the number z into an output.

Rosenblatt's choice, 1958

φ is the **step function**: 1 if $z \geq 0$, else 0. The model is then a **half-plane** — everything on one side of the line $w^\top x + b = 0$ is class 1.

You have met this object twice already, wearing different activations: with $\varphi(z) = z$ it is **linear regression**; with $\varphi(z) = \sigma(z)$ it is **logistic regression**. The perceptron is the same unit with a hard threshold.

The perceptron learning rule

$$w \leftarrow w + \eta(y - \hat{y})x, \quad b \leftarrow b + \eta(y - \hat{y})$$

Read the three cases

$y - \hat{y} = 0$: correct, **change nothing**.

$y - \hat{y} = +1$: we said 0, truth is 1. Add ηx , which *raises* $w^\top x$ on this row.

$y - \hat{y} = -1$: we said 1, truth is 0. Subtract ηx , which *lowers* it.

Mistake-driven and local

The rule looks at **one row at a time** and only acts when it is wrong. There is no loss function being minimised here — the step function has zero derivative everywhere it is differentiable, so gradient descent has nothing to descend.

Everything after the next section replaces this rule with one that *is* a gradient. But the shape — *error* $\times$ *input* — survives all the way to the last slide.

Predict first #1 — learning AND

Commit to two numbers

Four rows: $(0, 0) \rightarrow 0$, $(0, 1) \rightarrow 0$, $(1, 0) \rightarrow 0$, $(1, 1) \rightarrow 1$. Start from $w = (0, 0)$, $b = 0$, $\eta = 1$, and sweep the four rows in order, over and over.

How many **epochs** until it stops making mistakes?

How many **weight updates** in total?

Predict first #1 — learning AND

Commit to two numbers

Four rows: $(0,0) \rightarrow 0$, $(0,1) \rightarrow 0$, $(1,0) \rightarrow 0$, $(1,1) \rightarrow 1$. Start from $w = (0,0)$, $b = 0$, $\eta = 1$, and sweep the four rows in order, over and over.

How many **epochs** until it stops making mistakes?

How many **weight updates** in total?

6 epochs, **11** weight updates.

Almost everyone guesses two or three. It takes six because **the very first row $(0,0)$ is misclassified**: $z = 0 \geq 0$ so the step function says 1, and the only way to fix that is to push b down — which then breaks the row it had already learned.

By hand: every update the rule makes

ep	x	y	z	yhat	err	w after	b after
1	(0,0)	0	+0.0	1	-1	(+0,+0)	-1
1	(0,1)	0	-1.0	0	+0	(+0,+0)	-1
1	(1,0)	0	-1.0	0	+0	(+0,+0)	-1
1	(1,1)	1	-1.0	0	+1	(+1,+1)	+0
2	(0,0)	0	+0.0	1	-1	(+1,+1)	-1
2	(0,1)	0	+0.0	1	-1	(+1,+0)	-2
2	(1,0)	0	-1.0	0	+0	(+1,+0)	-2
2	(1,1)	1	-1.0	0	+1	(+2,+1)	-1
3	(0,0)	0	-1.0	0	+0	(+2,+1)	-1
3	(0,1)	0	+0.0	1	-1	(+2,+0)	-2
3	(1,0)	0	+0.0	1	-1	(+1,+0)	-3
3	(1,1)	1	-2.0	0	+1	(+2,+1)	-2
4	(0,0)	0	-2.0	0	+0	(+2,+1)	-2
4	(0,1)	0	-1.0	0	+0	(+2,+1)	-2
4	(1,0)	0	+0.0	1	-1	(+1,+1)	-3
4	(1,1)	1	-1.0	0	+1	(+2,+2)	-2
5	(0,0)	0	-2.0	0	+0	(+2,+2)	-2
5	(0,1)	0	+0.0	1	-1	(+2,+1)	-3
5	(1,0)	0	-1.0	0	+0	(+2,+1)	-3
5	(1,1)	1	+0.0	1	+0	(+2,+1)	-3
6	(0,0)	0	-3.0	0	+0	(+2,+1)	-3
6	(0,1)	0	-2.0	0	+0	(+2,+1)	-3
6	(1,0)	0	-1.0	0	+0	(+2,+1)	-3
6	(1,1)	1	+0.0	1	+0	(+2,+1)	-3

converged after 6 epochs and 11 weight updates
final rule: predict 1 when $2 \cdot x_1 + 1 \cdot x_2 - 3 \geq 0$
boundary line: $x_2 = 3.0000 - 2.0000 \cdot x_1$

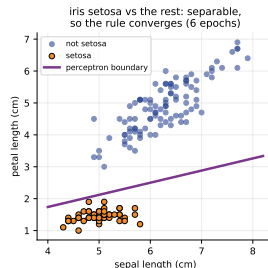
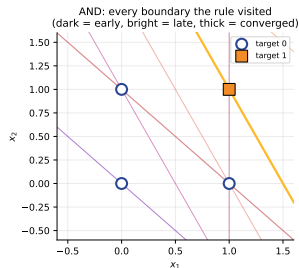
Reading that table, and drawing it

Where the eleven updates went

Six are -1 corrections (we said 1, truth 0), five are $+1$. Epoch 5 makes one update and epoch 6 makes none: **zero updates in a full sweep is the stopping condition**.

The answer is a line

$2x_1 + x_2 - 3 \geq 0$. Check the corners: $-3, -2, -1, 0$. Only the last is ≥ 0 . It stops *on* the boundary — the rule has no notion of margin, which is the gap SVMs exist to fill.



Left: the twelve weight states on AND, dark early, bright late; the thick orange line is the converged rule. Right: the same rule on load_iris, setosa against the other two — separable, so it converges, in six epochs again.

The same thing in code, on a real dataset

```
def perceptron_fit(X, y, epochs=100, eta=1.0):  
    w, b = np.zeros(X.shape[1]), 0.0  
    for ep in range(epochs):  
        errors = 0  
        for k in range(len(X)):  
            yh = 1.0 if (w @ X[k] + b) >= 0 else 0.0  
            err = y[k] - yh  
            if err != 0:  
                w, b, errors = w + eta*err*X[k], b + eta*err, errors + 1  
        if errors == 0: return w, b, ep + 1
```

```
iris setosa vs rest, 150 rows, 2 features  
epoch 1 updates 1 training accuracy 0.6667  
epoch 4 updates 2 training accuracy 0.6667  
epoch 5 updates 1 training accuracy 1.0000  
epoch 6 updates 0 training accuracy 1.0000  
converged in 6 epochs, final training accuracy 1.0000  
final weights w = [+3.50 -9.20] b = +2.00  
sklearn Perceptron on the same data: accuracy 1.0000
```

Eight lines. `sklearn.linear_model.Perceptron` is the same rule with bookkeeping.

What the rule guarantees — and what it does not

Perceptron convergence theorem

If the two classes are **linearly separable**, the rule reaches zero training errors in a finite number of updates, bounded by $(R/\gamma)^2$ where R is the largest $\|x\|$ and γ the margin of the best separator. It does not depend on η , on the order of the rows, or on where you start.

And if they are not separable?

The theorem says nothing, and neither does the algorithm. It does not converge, it does not warn you, and it does not return the best line it saw. It **cycles forever**.

There is no “stop when the error stops improving” in the rule as Rosenblatt stated it. That is not a small print issue — it is the subject of the next section.

Predict first #2 — exclusive OR

Commit to a number

Four rows again: $(0, 0) \rightarrow 0$, $(0, 1) \rightarrow \mathbf{1}$, $(1, 0) \rightarrow \mathbf{1}$, $(1, 1) \rightarrow 0$. Output 1 when the inputs *differ*.

Run the perceptron rule for a thousand epochs.

What training accuracy does it end on?
And what is the best accuracy *any* straight line can get?

Predict first #2 — exclusive OR

Commit to a number

Four rows again: $(0, 0) \rightarrow 0$, $(0, 1) \rightarrow \mathbf{1}$, $(1, 0) \rightarrow \mathbf{1}$, $(1, 1) \rightarrow 0$. Output 1 when the inputs *differ*.

Run the perceptron rule for a thousand epochs.

What training accuracy does it end on?
And what is the best accuracy *any* straight line can get?

The rule ends on **0.50**. The best line achieves **0.75**.

Two separate failures. No line gets all four — and the learning rule does not even find the best line, because it never stops correcting.

Why no line works

The geometry, in one sentence

A line cuts the plane into two half-planes. The two 1s sit at $(0, 1)$ and $(1, 0)$ — **diagonally opposite corners** of the unit square. Any half-plane containing both must also contain the midpoint $(0.5, 0.5)$, and hence some of the segment joining the two 0s.

The algebra, in four inequalities

Suppose $w_1x_1 + w_2x_2 + b \geq 0$ exactly on the 1s:

$$b < 0 \quad \text{from } (0, 0) \rightarrow 0$$

$$w_2 + b \geq 0 \quad \text{from } (0, 1) \rightarrow 1$$

$$w_1 + b \geq 0 \quad \text{from } (1, 0) \rightarrow 1$$

$$w_1 + w_2 + b < 0 \quad \text{from } (1, 1) \rightarrow 0$$

Add rows 2 and 3: $w_1 + w_2 + 2b \geq 0$. Row 4 says $w_1 + w_2 + b < 0$, so $b > 0$.

Contradiction with row 1.

Minsky and Papert, 1969. This one page stopped neural-network funding for a decade.

The same thing in code

```
w, b = np.zeros(2), 0.0
for ep in range(1000):
    for k in range(4):
        yh = 1.0 if (w @ X_xor[k] + b) >= 0 else 0.0
        w, b = w + (y_xor[k]-yh)*X_xor[k], b + (y_xor[k]-yh)
```

```
perceptron on XOR, 1000 epochs
best accuracy ever reached      0.50
accuracy after the last epoch   0.50
distinct (w1, w2, b) states visited: 2 -- it cycles, it does not settle
sklearn Perceptron, 10000 iterations: accuracy 0.50
logistic regression (also a single linear unit): accuracy 0.50
exhaustive search over  $81^3 = 531441$  lines: the best any line achieves
on XOR is 0.75 (3 of 4 rows), e.g.  $w = (-4.0, -4.0)$ ,  $b = +4.0$ 
```

Two end-of-epoch states, visited alternately, for a thousand epochs. Logistic regression — which *is* differentiable and *does* minimise a loss — gets 0.50 too, because it is still one linear unit.

By hand: two hidden units solve it

Build them from what we have

$h_1 = \text{OR}(x_1, x_2)$: weights (1, 1), bias -0.5 .

$h_2 = \text{AND}(x_1, x_2)$: weights (1, 1), bias -1.5 .

Then $\hat{y} = \text{OR}$ *and not* AND: weights (+1, -1) on (h_1, h_2) , bias -0.5 .

x1	x2	h1=OR	h2=AND	y_hat	target
0	0	0	0	0	0
0	1	1	0	1	1
1	0	1	0	1	1
1	1	1	1	0	0

accuracy 1.00

hidden representation of the four inputs:

(0,0) $\rightarrow$ (0,0)

(0,1) $\rightarrow$ (1,0)

(1,0) $\rightarrow$ (1,0)

(1,1) $\rightarrow$ (1,1)

Every one of these three units is a perceptron. Nothing new was invented — they were only **stacked**.

The hidden layer is a change of coordinates

$(0, 1)$ and $(1, 0)$ both land on $(1, 0)$.

What that buys

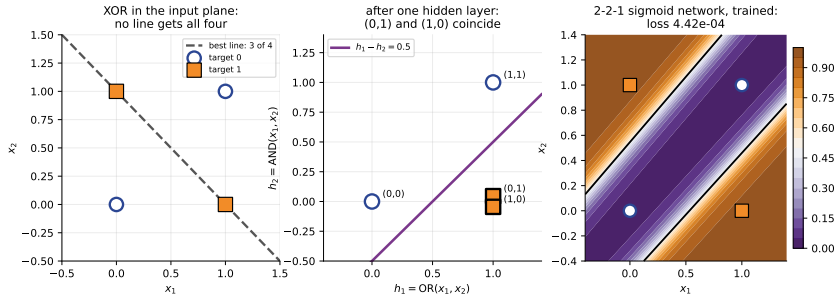
In the input plane the two positives are diagonally opposite. In the hidden plane they are **the same point**. Three distinct points remain: $(0, 0)$, $(1, 0)$, $(1, 1)$ — and the middle one is the only positive. One line separates it from the other two.

The general statement

A hidden layer does not classify. It **re-represents**: it maps the input into a new space chosen so that the last layer's linear model works there. Everything a deep network does is this, repeated.

The output layer is always just logistic (or linear) regression. The layers below it are learning the features that regression

Drawn: XOR, before and after the hidden layer



Left: the best line, three of four. Middle: after the hidden layer, the two positives coincide and a line does exist. Right: the decision surface of a trained 2–2–1 sigmoid network, final loss 3.30×10^{-4} .

And the network finds those units for itself

```
2-2-1 sigmoid network trained on XOR, 20000 steps of full-batch descent
loss at step      0: 0.125700
loss at step 1000: 0.125008
loss at step 20000: 0.000442
outputs [0.0299 0.9717 0.9663 0.0265] against targets [0 1 1 0]
accuracy 1.00
learned hidden weights
  unit 1: w = [-5.80 +5.91]  b = +2.96
  unit 2: w = [-5.37 +5.17]  b = -2.81
output layer: w = [-8.17 +8.66]  b = +3.80
hidden units 1 -> final loss 0.118611
hidden units 2 -> final loss 0.000442
```

It did *not* learn OR and AND. Unit 2 fires on “ x_2 and not x_1 ”; unit 1 fires everywhere *except* (1,0) and the output weight on it is -8.17 , so what reaches the output is “ x_1 and not x_2 ” upside down.

Same decomposition, one half inverted — and with **one** hidden unit the loss stalls at 0.1186, barely below its starting 0.1257.

The feedforward network, in general

$$a^{(0)} = x, \quad z^{(\ell)} = W^{(\ell)\top} a^{(\ell-1)} + b^{(\ell)}, \quad a^{(\ell)} = \varphi^{(\ell)}(z^{(\ell)})$$

The vocabulary the exam uses

Feedforward — the graph has no cycles; information moves input to output, once.

Fully connected / dense — every unit in layer ℓ reads every unit in $\ell - 1$.

Multi-layer perceptron (MLP) — a feedforward, fully connected network with at least one hidden layer and a *differentiable* activation.

An MLP is not made of perceptrons

The name is historical and misleading. A perceptron has a step activation; an MLP must not, because backpropagation needs a derivative. Say “unit” or “neuron”, not “perceptron”, when you describe a hidden layer.

Counting parameters

A 64–32–10 network has

Take the activation away and the network collapses

Three linear layers

$$f(x) = ((xW_1 + b_1)W_2 + b_2)W_3 + b_3 = x \underbrace{W_1 W_2 W_3}_{W_{\text{eff}}} + \underbrace{b_1 W_2 W_3 + b_2 W_3 + b_3}_{b_{\text{eff}}}$$

The proof is one line of algebra

A product of matrices is a matrix. A composition of affine maps is an affine map. **Depth without non-linearity buys nothing at all.**

And the rank bound is worse

$W_{\text{eff}} = W_1 W_2 W_3$ is 4×2 , so its rank is at most $\min(4, 6, 5, 2) = 2$. A narrow middle layer *caps* what the whole stack can express, however wide the rest is.

The same thing in code, then on real data

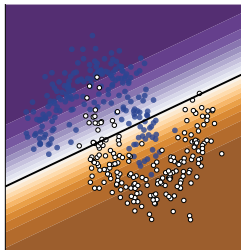
```
deep = ((X @ W1 + b1) @ W2 + b2) @ W3 + b3
W_eff = W1 @ W2 @ W3
b_eff = b1 @ W2 @ W3 + b2 @ W3 + b3
flat = X @ W_eff + b_eff
print(np.abs(deep - flat).max())
```

```
three linear layers 4->6->5->2, no activation
parameters in the deep version : 77
parameters in the flat version : 10
max |deep(x) - flat(x)| over 7 inputs: 7.105e-15
W_eff = W1 W2 W3 has shape (4, 2) and rank 2
moons, 400 points
logistic regression (1 linear unit)      accuracy 0.8750
MLP 32-32 with identity activation         accuracy 0.8775
MLP 32-32 with relu activation            accuracy 0.9625
1058 parameters bought the identity network +0.0025 over 3 parameters
```

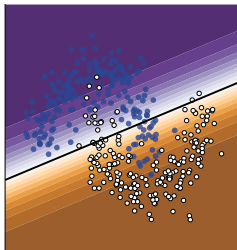
7.1×10^{-15} is floating-point noise: the two functions are the same function. And 1058 identity-activated parameters beat 3 by 0.0025 — which is one point out of four hundred.

Drawn: what the activation is worth

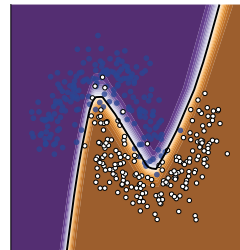
logistic regression
3 parameters
accuracy 0.8750



MLP 32-32, identity
1058 parameters
accuracy 0.8775



MLP 32-32, ReLU
1058 parameters
accuracy 0.9625



Same architecture, same data, same optimiser. The middle panel has 1058 parameters and draws a straight line, because that is all it can draw. Swap identity for relu and the boundary bends.

The three activations, and the derivatives backprop need

Definitions and derivatives

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z))$$

$$\tanh z, \quad \tanh' z = 1 - \tanh^2 z$$

$$\text{ReLU}(z) = \max(0, z),$$

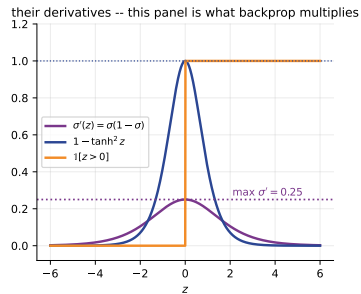
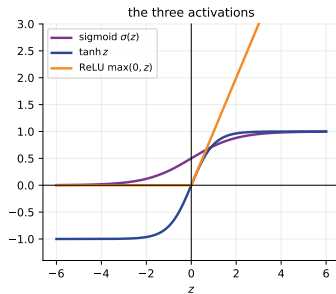
$$\text{ReLU}'(z) = \mathbf{1}[z > 0]$$

Note what the first two have in common:
the derivative is expressible **in the output**.
Backprop saves a, never z.

z	sigmoid	s'(z)	tanh	t'(z)	relu
-3.0	0.0474	0.0452	-0.9951	0.0099	0.0
-1.0	0.2689	0.1966	-0.7616	0.4200	0.0
+0.0	0.5000	0.2500	+0.0000	1.0000	0.0
+1.0	0.7311	0.1966	+0.7616	0.4200	1.0
+3.0	0.9526	0.0452	+0.9951	0.0099	3.0
max of sigmoid' 0.250000 at z = +0.0000 max of tanh' 1.000000 at z = +0.0000 sigmoid'(z) < 0.01 whenever z > 4.5849					

σ' never exceeds **0.25**. $\tanh'$ never exceeds **1**. ReLU' is **1** or **0** — a gate, not a scale.

Drawn: the functions, and the panel backprop multiplies



The left panel is what the forward pass computes. **The right panel is what the backward pass multiplies by**, once per layer. Read the dotted line at 0.25 and then look at the next slide's question.

Predict first #3 — ten sigmoid layers

Commit to an order of magnitude

Ten layers, sixteen units wide, **identical weights** in both networks. One stack uses sigmoid, the other ReLU. Push the same batch through, take the same loss, and measure $\|\partial L / \partial W^{(1)}\|$ — the gradient reaching the layer nearest the input.

What is the ratio, ReLU over sigmoid?

Predict first #3 — ten sigmoid layers

Commit to an order of magnitude

Ten layers, sixteen units wide, **identical weights** in both networks. One stack uses sigmoid, the other ReLU. Push the same batch through, take the same loss, and measure $\|\partial L / \partial W^{(1)}\|$ — the gradient reaching the layer nearest the input.

What is the ratio, ReLU over sigmoid?

1.9017×10^{-2} against 6.9609×10^{-7} : a factor of **27 320**.

And it is not a mystery. $0.25^{10} = 9.54 \times 10^{-7}$; the measured attenuation from layer 10 down to layer 1 was **8.32×10^{-7}** . The arithmetic predicted it.

The measurement, layer by layer

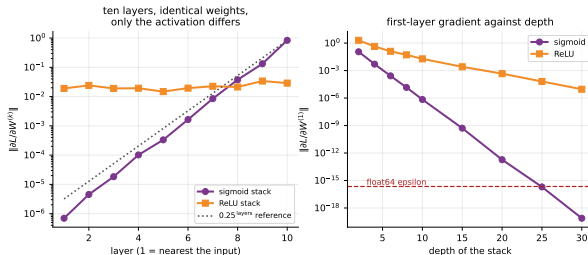
ten layers of width 16, identical weights, only the activation differs

layer	dL/dW sigmoid	dL/dW relu	ratio relu/sigmoid
1	6.9609e-07	1.9017e-02	27320.0
2	4.5222e-06	2.4150e-02	5340.4
3	1.8296e-05	1.8996e-02	1038.2
5	3.3209e-04	1.4784e-02	44.5
8	3.7576e-02	2.1301e-02	0.6
10	8.3697e-01	2.8907e-02	0.0

attenuation across the sigmoid stack, layer 10 -> layer 1: 8.3167e-07
attenuation across the relu stack, layer 10 -> layer 1: 6.5787e-01
for comparison, $0.25^{**9} = 3.8147e-06$ and $0.25^{**10} = 9.5367e-07$

Read the sigmoid column upwards: it falls by roughly a factor of three to six **per layer**, compounding. The ReLU column is flat — it ends at 0.658 of where it started, not 10^{-7} . At layer 10 the sigmoid gradient is nearly thirty times the *larger* of the two; the damage is entirely in the propagation.

Drawn: depth against the surviving gradient



depth 2	sigmoid layer-1 norm	1.129e-01	relu layer-1 norm	2.009e+00
depth 10	sigmoid layer-1 norm	6.961e-07	relu layer-1 norm	1.902e-02
depth 30	sigmoid layer-1 norm	7.569e-20	relu layer-1 norm	9.209e-06

A straight line on a log axis is geometric decay. Thirty sigmoid layers put the first-layer gradient at 10^{-20} — far below anything single precision can represent against a weight of order one.

And this is not a thought experiment

30 epochs, learning rate 0.1, hidden layers of 32 units each, load_digits

activation	hidden layers	train loss	test accuracy
relu	1	0.0758	0.9711
relu	3	0.0114	0.9733
relu	6	0.0030	0.9667
relu	10	0.2811	0.8778
sigmoid	1	0.3303	0.9422
sigmoid	3	2.3062	0.1000
sigmoid	6	2.3227	0.1000
sigmoid	10	2.3184	0.1000

Chance on ten balanced classes is 0.1000, and a loss of $\ln 10 = 2.3026$ is the loss of a network that outputs a uniform distribution. **Three sigmoid hidden layers do not train at all here** — not badly, at all. ReLU is fine to six layers and starts to strain at ten, which is an honest result and the reason residual connections were invented.

Backpropagation is the chain rule, and nothing else

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial x}$$

Read the product from the left

For $x \rightarrow h_1 \rightarrow h_2 \rightarrow y \rightarrow L$,

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial x}.$$

Evaluate it **right to left** and you need one pass per input: a million parameters, a million passes. Evaluate it **left to right**, starting from the single number L , and

Each layer's job is local

A layer is handed $\partial L / \partial(\text{its output})$ and must return $\partial L / \partial(\text{its input})$ and $\partial L / \partial(\text{its parameters})$. It supplies only its **own** derivative and multiplies through.

It does not know what is above it or below it. That locality is why you can write a layer once, test it once, and compose it forever.

The network we are going to differentiate

Two inputs, two hidden, one output

$$x_1 = 1.0, \quad x_2 = 2.0, \quad \text{target } t = 1.0$$

$$z_1 = w_{11}x_1 + w_{21}x_2 + b_{11}, \quad a_1 = \sigma(z_1)$$

$$z_2 = w_{12}x_1 + w_{22}x_2 + b_{12}, \quad a_2 = \sigma(z_2)$$

$$z_3 = v_1a_1 + v_2a_2 + b_2, \quad a_3 = \sigma(z_3)$$

$$L = \frac{1}{2}(a_3 - t)^2$$

The nine parameters

$$w_{11} \quad +0.2 \quad w_{12} \quad +0.3$$

$$w_{21} \quad -0.1 \quad w_{22} \quad +0.1$$

$$b_{11} \quad 0.0 \quad b_{12} \quad +0.5$$

$$v_1 \quad +0.4 \quad v_2 \quad -0.6$$

$$b_2 \quad +0.1$$

Nine parameters is small enough to do entirely with a pen and large enough that every case appears: two layers, a shared downstream path, and biases. **Copy these numbers down.**

Forward pass, by hand

$$z_1 = (0.2)(1.0) + (-0.1)(2.0) + 0.0 = 0.0$$

$$a_1 = \sigma(0) = \mathbf{0.5000000000}$$

$$z_2 = (0.3)(1.0) + (0.1)(2.0) + 0.5 = 1.0$$

$$a_2 = \sigma(1) = \mathbf{0.7310585786}$$

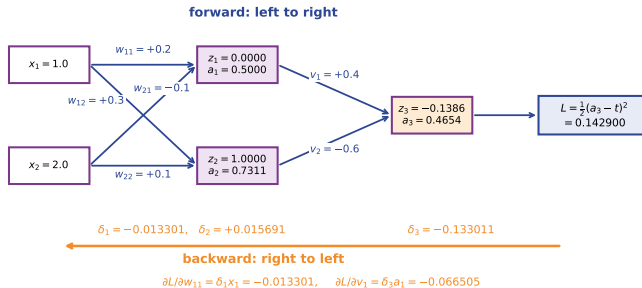
$$z_3 = (0.4)(0.5) + (-0.6)(0.7310585786) + 0.1 = -\mathbf{0.1386351472} \quad a_3 = \sigma(z_3) = \mathbf{0.4653966177}$$

$$L = \frac{1}{2}(0.4653966177 - 1)^2 = \frac{1}{2}(-0.5346033823)^2 = \mathbf{0.1429003882}$$

```
z1 = 0.2*1.0 + -0.1*2.0 + 0.0 = +0.0000    a1 = sigma(z1) = 0.5000000000
z2 = 0.3*1.0 + 0.1*2.0 + 0.5 = +1.0000    a2 = sigma(z2) = 0.7310585786
z3 = 0.4*0.5000000000 + -0.6*0.7310585786 + 0.1 = -0.1386351472
a3 = sigma(z3) = 0.4653966177
L = 0.5 * (0.4653966177 - 1.0)^2 = 0.1429003882
```

The two weights were chosen so $z_1 = 0$ exactly: $\sigma(0) = 0.5$ and $\sigma'(0) = 0.25$, both by hand.

Drawn: the same network, forward and backward



Every number on this figure was produced by the code, not typed onto it. The orange arrow is the direction of the next four slides.

Backward, step 1: the output unit

Two factors, then their product

$$\frac{\partial L}{\partial a_3} = a_3 - t = 0.4653966177 - 1 = -\mathbf{0.5346033823}$$

$$\frac{\partial a_3}{\partial z_3} = a_3(1 - a_3) = (0.4653966177)(0.5346033823) = +\mathbf{0.2488026059}$$

$$\delta_3 \equiv \frac{\partial L}{\partial z_3} = \frac{\partial L}{\partial a_3} \cdot \frac{\partial a_3}{\partial z_3} = (-0.5346033823)(0.2488026059) = -\mathbf{0.1330107147}$$

δ_3 is the whole of the output layer's message to everything below. **One number.** Every gradient in the network is δ_3 multiplied by something local.

Backward, step 2: the output layer's own weights

Because $z_3 = v_1 a_1 + v_2 a_2 + b_2$

$$\frac{\partial L}{\partial v_1} = \delta_3 \cdot \frac{\partial z_3}{\partial v_1} = \delta_3 a_1 = (-0.1330107147)(0.5000000000) = -\mathbf{0.0665053573}$$

$$\frac{\partial L}{\partial v_2} = \delta_3 a_2 = (-0.1330107147)(0.7310585786) = -\mathbf{0.0972386240}$$

$$\frac{\partial L}{\partial b_2} = \delta_3 \cdot 1 = -\mathbf{0.1330107147}$$

The pattern, already visible

gradient of a weight = **delta** × **the activation**

A bias multiplies the constant 1, so its gradient is the delta itself. That is the

Backward, step 3: push the delta through the hidden layer

The recursion — this is the line that makes it “back”-propagation

$$\delta_1 \equiv \frac{\partial L}{\partial z_1} = \underbrace{\delta_3}_{\text{from above}} \cdot \underbrace{v_1}_{\text{the path}} \cdot \underbrace{a_1(1 - a_1)}_{\sigma'(z_1)} = (-0.1330107147)(0.4)(0.25) = -\mathbf{0.0133010715}$$

$$\begin{aligned} \delta_2 \equiv \frac{\partial L}{\partial z_2} &= \delta_3 \cdot v_2 \cdot a_2(1 - a_2) \\ &= (-0.1330107147)(-0.6)(0.7310585786)(0.2689414214) = +\mathbf{0.0156908963} \end{aligned}$$

Three things happened to the message: it was **routed** along the weight it came down (v_j), and **throttled** by the local derivative (σ'). Note δ_2 has the *opposite sign* to δ_3 , because v_2 is negative. And $a_1(1 - a_1) = 0.25$ exactly: this unit sits at the steepest point of the sigmoid, the most gradient it can ever pass.

Backward, step 4: the input layer's weights

Same rule as step 2, one layer down: $\delta \times \text{input}$

$$\frac{\partial L}{\partial w_{11}} = \delta_1 x_1 = (-0.0133010715)(1.0) = -0.0133010715$$

$$\frac{\partial L}{\partial w_{21}} = \delta_1 x_2 = (-0.0133010715)(2.0) = -0.0266021429$$

$$\frac{\partial L}{\partial b_{11}} = \delta_1 = -0.0133010715$$

$$\frac{\partial L}{\partial w_{12}} = \delta_2 x_1 = +0.0156908963$$

$$\frac{\partial L}{\partial w_{22}} = \delta_2 x_2 = +0.0313817925$$

$$\frac{\partial L}{\partial b_{12}} = \delta_2 = +0.0156908963$$

$\partial L / \partial w_{21}$ is exactly twice $\partial L / \partial w_{11}$ because $x_2 = 2x_1$. **An input twice as large gets a gradient twice as large** — which is why you scale your features before training a network.

The two rules, stated once, for any depth

$$\delta^{(\ell)} = (W^{(\ell+1)}\delta^{(\ell+1)}) \odot \varphi'(z^{(\ell)})$$

$$\frac{\partial L}{\partial W^{(\ell)}} = a^{(\ell-1)}\delta^{(\ell)\top}$$

That is the entire algorithm

Start at the top with $\delta^{(L)} = \nabla_a L \odot \varphi'(z^{(L)})$. Apply the left rule to walk down. Apply the right rule at each layer to read off the gradients. Add $\partial L / \partial b^{(\ell)} = \delta^{(\ell)}$ and you are finished.

Two shapes to get right

The $\odot$ is **elementwise**: the activation derivative is a diagonal Jacobian, so it never becomes a matrix product. And for a batch of n rows, the bias gradient is the **sum over the batch**, because one bias was copied n times. That summation is the single commonest bug in a hand-written backward pass.

Predict first #4 — is the derivation right?

Commit to an exponent

We now have nine numbers computed with a pen. Check them the dumb way: for each parameter p , nudge it by $h = 10^{-6}$ each way, run the forward pass twice, and take $(L(p+h) - L(p-h))/2h$.

How closely will the two agree?
(Give it as a power of ten.)

Predict first #4 — is the derivation right?

Commit to an exponent

We now have nine numbers computed with a pen. Check them the dumb way: for each parameter p , nudge it by $h = 10^{-6}$ each way, run the forward pass twice, and take $(L(p+h) - L(p-h))/2h$.

How closely will the two agree?
(Give it as a power of ten.)

Largest disagreement over all nine: 4.107×10^{-11} .

And that residue is the *finite difference's* error, not backpropagation's. The central difference is accurate to $O(h^2)$, and with $h = 10^{-6}$ that is $O(10^{-12})$, degraded a little by cancellation in the subtraction.

Verified once: central finite differences

$h = 1e-6$

```
for k in base: # base = the nine parameters
    up, dn = dict(base), dict(base)
    up[k], dn[k] = base[k] + h, base[k] - h
    fd = (tiny_loss(up) - tiny_loss(dn)) / (2 * h)
    print(k, hand[k], fd, abs(fd - hand[k]))
```

parameter	hand-computed	finite difference	difference
w11	-0.013301071466	-0.013301071425	4.107e-11
w12	+0.015690896251	+0.015690896291	4.005e-11
w21	-0.026602142932	-0.026602142905	2.662e-11
w22	+0.031381792501	+0.031381792540	3.846e-11
b11	-0.013301071466	-0.013301071425	4.107e-11
b12	+0.015690896251	+0.015690896291	4.005e-11
v1	-0.066505357329	-0.066505357305	2.492e-11
v2	-0.097238624001	-0.097238623986	1.502e-11
b2	-0.133010714659	-0.133010714679	1.954e-11

largest disagreement over all nine parameters: 4.107e-11

Nine for nine, to ten significant figures. **Do this every time you write a backward pass by hand.** A gradient that is wrong by a factor of two still trains, just worse, and you will blame the learning rate for a week.

Verified twice: autodiff, in a library you can open

```
import gradkit as gk
W1g = gk.tensor(W1, requires_grad=True); b1g = gk.tensor(b1, requires_grad=True)
)
W2g = gk.tensor(W2.reshape(2,1), requires_grad=True); b2g = gk.tensor([b2],
requires_grad=True)
a1g = gk.sigmoid(gk.tensor(x.reshape(1,2)) @ W1g + b1g)
a2g = gk.sigmoid(a1g @ W2g + b2g)
Lg = ((a2g - tg) * (a2g - tg)).sum() * 0.5
Lg.backward()
```

```
gradkit forward loss 0.142900388188 (hand: 0.142900388188)
parameter    hand          gradkit      |difference|
w11          -0.013301071466 -0.013301071466 0.000e+00
w21          -0.026602142932 -0.026602142932 0.000e+00
v1           -0.066505357329 -0.066505357329 0.000e+00
b2           -0.133010714659 -0.133010714659 0.000e+00
largest hand-vs-autodiff disagreement: 0.000e+00
largest hand-vs-finite-difference disagreement: 4.107e-11
all nine hand-computed gradients confirmed twice
```

Zero, bit for bit — because autodiff evaluates the same product of the same local derivatives, in the same order. gradkit is written for this course; open `functions/activation.py` and you will find $\sigma(1 - \sigma)$ exactly as we wrote it. <https://github.com/tali7c/gradkit>

What you do with nine gradients

```
one step of gradient descent, p <- p - eta * dL/dp
eta  0.5  loss 0.1429003882 -> 0.1263134462  (-1.66e-02)
eta  5.0  loss 0.1429003882 -> 0.0297255097  (-1.13e-01)
eta 50.0  loss 0.1429003882 -> 0.0000000004  (-1.43e-01)
after  1 steps at eta=5.0: loss 0.02972551  output 0.756174
after 10 steps at eta=5.0: loss 0.00262249  output 0.927578
after 100 steps at eta=5.0: loss 0.00021488  output 0.979270
after 500 steps at eta=5.0: loss 0.00003898  output 0.991170
```

The output climbs $0.4654 \rightarrow 0.7562 \rightarrow 0.9912$ towards the target 1.0. Do not read too much into $\eta = 50$ working here: this is *one* training row and one row can always be fitted. On a real dataset that learning rate diverges immediately.

One special case worth memorising: softmax with cross

With $p = \text{softmax}(z)$ and $L = -\sum_c y_c \log p_c$, the softmax Jacobian $p_i(\delta_{ij} - p_j)$ and the $1/p$ from the logarithm cancel exactly, leaving

$$\frac{\partial L}{\partial z} = p - y.$$

```
logits z          = [2.0 1.0 0.1]
softmax p         = [0.659001 0.242433 0.098566]
target one-hot y  = [0 1 0]
L = -log p_1      = 1.417030
claim: dL/dz = p - y = [+0.659001 -0.757567 +0.098566]
finite difference = [+0.659001 -0.757567 +0.098566]
largest disagreement = 2.168e-10
```

So the output layer of a classifier needs **no activation derivative at all** in code: the delta is just (probs - onehot). That is why the NumPy network on the next slides starts its backward pass with one subtraction.

The algorithm in eight lines

For each mini-batch

1. $a^{(0)} \leftarrow X_{\text{batch}}$
2. for $\ell = 1 \dots L$: $z^{(\ell)} = a^{(\ell-1)}W^{(\ell)} + b^{(\ell)}$, $a^{(\ell)} = \varphi(z^{(\ell)})$ *forward*
3. $\delta^{(L)} \leftarrow (a^{(L)} - Y)/n$ *softmax + cross-entropy*
4. for $\ell = L \dots 1$:
5. $\nabla W^{(\ell)} = a^{(\ell-1)\top} \delta^{(\ell)}$, $\nabla b^{(\ell)} = \sum_{\text{rows}} \delta^{(\ell)}$ *read off*
6. $\delta^{(\ell-1)} = (\delta^{(\ell)} W^{(\ell)\top}) \odot \varphi'(z^{(\ell-1)})$ *propagate*
7. $W^{(\ell)} \leftarrow W^{(\ell)} - \eta \nabla W^{(\ell)}$ for every ℓ *update*
8. repeat over all batches; that is one **epoch**

Lines 5 and 6 are the two rules from four slides ago, in matrix form. Nothing else has been added.

Written out in NumPy — the whole thing

```
def forward(P, X):
    A, Z = [X], []
    for k, (W, b) in enumerate(P):
        z = A[-1] @ W + b; Z.append(z)
        A.append(softmax(z) if k == len(P)-1 else relu(z))
    return A, Z

def backward(P, A, Z, Y):
    n, grads = len(Y), [None] * len(P)
    d = (A[-1] - Y) / n # softmax + cross-entropy, combined
    for k in range(len(P)-1, -1, -1):
        grads[k] = [A[k].T @ d, d.sum(0)]
        if k > 0:
            d = (d @ P[k][0].T) * (Z[k-1] > 0)
    return grads
```

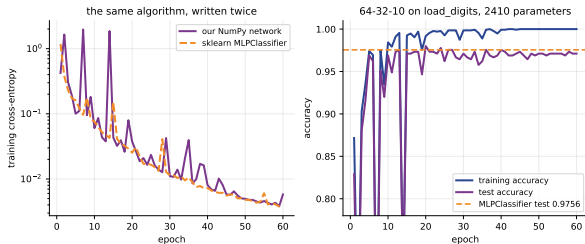
Fourteen lines, no framework. $A[k].T @ d$ is $a^{(\ell-1)\top} \delta^{(\ell)}$; $d.sum(0)$ is the bias summation over the batch; $(Z[k-1] > 0)$ is ReLU' .

Trained on load_digits

```
load_digits: 1797 rows x 64 pixels, 10 classes
train 1347 rows, test 450 rows, pixels scaled to [0, 1]
architecture 64 -> 32 -> 10, ReLU, softmax + cross-entropy
parameters: 64*32 + 32 + 32*10 + 10 = 2410
epoch   train loss   train acc   test acc
    1         0.4232       0.8716     0.8289
    5         0.1002       0.9740     0.9689
   10         0.0604       0.9844     0.9689
   20         0.0374       0.9918     0.9800
   40         0.0081       0.9993     0.9756
   60         0.0058       1.0000     0.9711
final test accuracy 0.9711
```

Read the last two rows together. Training accuracy reaches **1.0000** and test accuracy *falls* from its peak of 0.9800 at epoch 20 to 0.9711 at epoch 60. That is overfitting, and it is the argument for early stopping.

Drawn: the loss curve, ours against the library's



```
test accuracy from scratch, NumPy : 0.9711
test accuracy MLPClassifier       : 0.9756 (difference +0.0044)
MLPClassifier with its default Adam solver: 0.9733
logistic regression, no hidden layer : 0.9689
```

Two independent implementations of the same fourteen lines, tracking each other for sixty epochs. The spikes are mini-batch noise at $\eta = 0.5$, present in both.

Gradient-check the whole network, not just the toy

64-32-10 network, batch of 8 rows, 60 randomly chosen weight entries

layer	entry	backprop	finite difference	diff
W1	(25,27)	-0.001717812303	-0.001717812292	1.02e-11
W1	(5,27)	-0.046692799725	-0.046692799716	9.74e-12
W1	(1,17)	-0.000164852261	-0.000164852265	3.69e-12
W1	(5, 9)	-0.035753919656	-0.035753919647	9.27e-12
W1	(29,31)	+0.036876849891	+0.036876849885	5.85e-12
largest absolute disagreement		: 2.796e-11		
largest relative error where grad >1e-6		: 2.240e-08		

Many sampled entries came back +0.000000000000 in *both* columns. Those are border pixels that are 0 in all eight rows, or dead ReLU units — genuinely no gradient, correctly reported as none.

And the same model in the course's own library

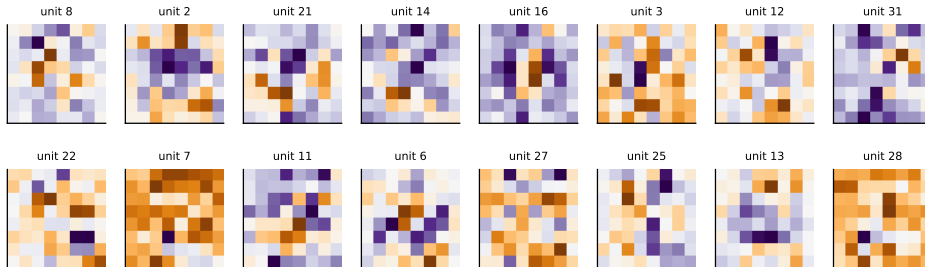
```
from gradkit import nn, optim
model = nn.Sequential(nn.Dense(64, 32), nn.ReLU(), nn.Dense(32, 10))
gk.fit(model, (Xtr, ytr), "cross_entropy",
        optim.SGD(model.parameters(), lr=0.5), epochs=60, batch_size=32)
```

```
test accuracy, gradkit          : 0.9667
test accuracy, our NumPy version : 0.9711
test accuracy, MLPClassifier     : 0.9756
three implementations of the same algorithm, three accuracies within
0.0089 of each other.
gradcheck(sigmoid) -> True
gradcheck(tanh    ) -> True
gradcheck(relu    ) -> True
```

A spread of 0.0089 on 450 test rows is **four rows**. Nothing separates the three implementations; they are the same algorithm with different shuffling. Use the library to *read*, not to depend on.

Drawn: what the hidden units actually learned

the 16 busiest hidden units of the trained 64-32-10 network, each drawn as the 8×8 image of its 64 input weights



Each tile is one hidden unit's 64 input weights, folded back into the 8×8 image shape. They are stroke and blob detectors — the features nobody specified, discovered because the gradient said so.

Predict first #5 — start everything at zero

Commit to a number

The same 64–32–10 network, the same data, the same thirty epochs. The only change: initialise **every weight to 0** instead of drawing them at random.

What test accuracy comes out?

Predict first #5 — start everything at zero

Commit to a number

The same 64–32–10 network, the same data, the same thirty epochs. The only change: initialise **every weight to 0** instead of drawing them at random.

What test accuracy comes out?

0.1000. Chance. Ten balanced classes, ten per cent.

And it is not slow learning that you could fix with more epochs. It is **structurally** stuck. All 32 hidden units compute the same thing, receive the same gradient, and take the same step — forever.

Symmetry, and why it never breaks

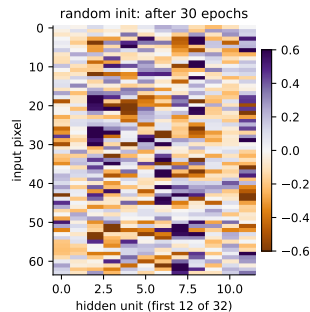
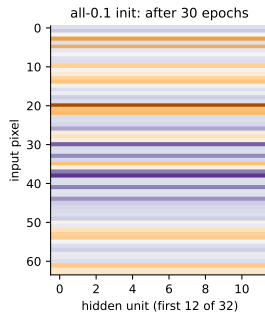
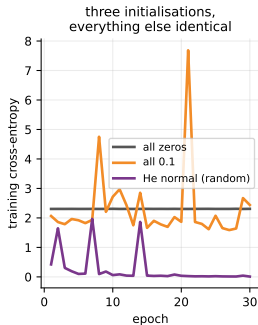
The argument

If two hidden units j and k start with identical incoming weights, they compute identical $a_j = a_k$ for every input. Their outgoing weights are identical too, so $\delta_j = \delta_k$, so $\nabla w_{ij} = \nabla w_{ik}$, so after the update they are *still* identical. By induction, forever.

```
30 epochs, identical architecture and data
init          train loss  train acc  test acc
all zeros      2.3081      0.1010      0.1000
all 0.1        2.4387      0.2517      0.2356
He normal      0.0110      0.9985      0.9711
after training, are the 32 units distinguishable?
zeros init : max spread across columns = 0.000e
+00
0.1 init : max spread across columns = 0.000e
+00
He init : max spread across columns = 2.774e
+00
zeros init : columns 0 and 17 identical? True
0.1 init : columns 0 and 17 identical? True
He init : columns 0 and 17 identical? False
```

It is not zero that is the problem, it is **equality**: all-0.1 fails the same way and reaches 0.2356. A 32-unit layer with identical columns is a 1-unit layer wearing a costume.

Drawn, and the rule to use



He: $\mathcal{N}(0, 2/n_{in})$ for ReLU.

Xavier/Glorot: $\mathcal{N}(0, 1/n_{in})$ for sigmoid and

Biases *may* start at zero — they are not what

Add one edge that points backwards

$$h_t = \varphi(W_{hh}h_{t-1} + W_{xh}x_t + b), \quad y_t = W_{hy}h_t$$

What changed

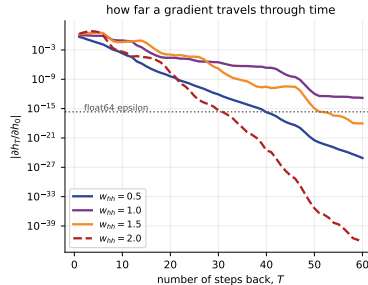
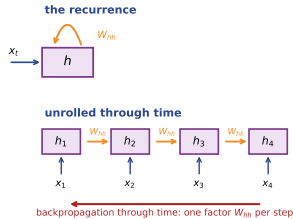
The layer's input now includes **its own previous output**. The network has a state, so its answer depends on the whole history, not just the current input. That is what makes it suitable for **sequences**: text, speech, sensor streams, log files.

And what did not

W_{hh} , W_{xh} , W_{hy} and b are the **same at every time step**. A hundred-step sequence does not have a hundred times as many parameters — it reuses one small set a hundred times, exactly as a for loop reuses one body.

Also called a **feedback** architecture, against the **feedforward** networks of the rest of this lecture.

Unrolling, and backpropagation through time



Unroll the loop over T steps and you have an ordinary feedforward network, T layers deep, whose layers share their weights. Backpropagation then applies unchanged — run it on the unrolled graph and *sum* each weight's gradient over the steps it appeared in. That is all **BPTT** is.

By hand: three steps, and the gradient back to the start

```
h_t = tanh(w_hh h_{t-1} + w_xh x_t + b),    x = (1, 0, 0), w_hh = 0.9, w_xh = 1
h1 = tanh(0.9 * 0.000000 + 1.0 * 1.0) = 0.761594
h2 = tanh(0.9 * 0.761594 + 1.0 * 0.0) = 0.595041
h3 = tanh(0.9 * 0.595041 + 1.0 * 0.0) = 0.489602
dh3/dh0 = prod_t w_hh (1 - h_t^2) = 0.150353
```

Where 0.150353 comes from

$$\frac{\partial h_T}{\partial h_0} = \prod_{t=1}^T w_{hh} (1 - h_t^2), \text{ and here } (0.9)(0.4200) \times (0.9)(0.6459) \times (0.9)(0.7603) = 0.150353.$$

A **product of T factors**. The input at $t = 1$ still echoes in $h_3 = 0.4896$, but the gradient that would teach the network to use it has already shrunk to 15%.

There are exactly two things that product can do

$|dh_T/dh_0|$ for a scalar tanh RNN

T	$w_{hh}=0.5$	$w_{hh}=1.0$	$w_{hh}=1.5$	$w_{hh}=2.0$
1	4.9803e-01	9.9606e-01	1.4941e+00	1.9921e+00
5	2.6534e-02	7.9059e-01	3.7933e+00	4.4321e+00
10	2.0326e-04	8.5842e-02	4.2845e-02	4.4101e-04
20	3.9998e-09	2.5262e-05	1.1633e-04	2.0875e-08
50	3.3054e-22	1.0712e-12	1.0959e-15	4.1206e-36
100	8.5708e-43	3.9559e-20	9.4559e-34	1.5963e-74

Small factors compound to zero: the **vanishing gradient**. Large factors compound to infinity: the **exploding gradient** — see $w_{hh} = 2.0$ more than doubling from 1.99 to 4.43 between $T = 1$ and $T = 5$ before tanh saturates and drives it back down. There is no third case; $|w(1 - h^2)| = 1$ exactly is a measure-zero accident. **Exploding is easy to fix** (clip the gradient norm). Vanishing is not, and that is what gates — LSTM, GRU — were invented for.

Measured: how far back an RNN can actually remember

task: read a binary sequence of length T , report its FIRST symbol.
one tanh recurrent unit bank of width 8, plain BPTT, 400 epochs.

T	held-out accuracy	$ \text{gradient at } t=1 $ at epoch 0
2	1.0000	$3.335\text{e-}03$
5	1.0000	$3.976\text{e-}04$
10	1.0000	$2.088\text{e-}05$
20	0.5500	$9.086\text{e-}08$
40	0.4925	$1.527\text{e-}12$

The task does not get harder with T — the answer is sitting at $t = 1$ every time, and the other symbols are pure distraction. The *only* thing that changes is how far the gradient has to travel.

At $T = 40$ it arrives at 10^{-12} and the network scores 0.4925: a coin. **This is the vanishing gradient measured on a task, not a diagram.**

Summary — fourteen lines

1. A unit is $\varphi(w^T x + b)$. With $\varphi = \text{identity}$ it is linear regression, with σ it is logistic regression, with a step it is a **perceptron**.
2. The perceptron rule $w \leftarrow w + \eta(y - \hat{y})x$ is mistake-driven. On AND it converges in **6** epochs and **11** updates to $2x_1 + x_2 - 3 \geq 0$; on separable iris, 6 epochs to accuracy 1.0000.
3. If the data is not separable the rule **cycles forever**. On XOR it ends at 0.50, revisiting two states; the best of all 531441 lines searched gets only **0.75**.
4. Two hidden units fix XOR by moving (0, 1) and (1, 0) to the same point. A hidden layer **re-represents**; the output layer is still just linear.
5. Without a non-linearity, depth is free of charge and worth nothing: three layers with 77 parameters equal one with 10 to **7.1×10^{-15}** , and on moons, 1058 identity parameters beat 3 by 0.0025 against ReLU's +0.0875.
6. $\sigma' \leq 0.25$, $\tanh' \leq 1$, $\text{ReLU}' \in \{0, 1\}$. Ten sigmoid layers attenuate the first-layer gradient to **6.96×10^{-7}** against ReLU's 1.90×10^{-2} ; a ratio of **27 320**, and $0.25^{10} = 9.54 \times 10^{-7}$ predicted it.
7. Consequence, measured: three sigmoid hidden layers on digits score **0.1000** — chance — while three ReLU layers score 0.9733.
8. Backpropagation is the chain rule read **left to right**: one forward and one backward pass give every parameter's derivative.
9. On the 2–2–1 network: $a_3 = 0.4653966177$, $L = \mathbf{0.1429003882}$, $\delta_3 = -0.1330107147$, $\delta_1 = -0.0133010715$, $\delta_2 = +0.0156908963$, and nine gradients from -0.1330 to $+0.0314$.
10. Two rules, any depth: $\delta^{(\ell)} = (W^{(\ell+1)} \delta^{(\ell+1)}) \odot \varphi'(z^{(\ell)})$ and $\nabla W^{(\ell)} = a^{(\ell-1)} \delta^{(\ell)T}$. For softmax with cross-entropy, $\partial L / \partial z = p - y$.
11. All nine hand gradients matched central differences to **4.107×10^{-11}** and autodiff to **0.000e+00**. The 64–32–10 network's gradients matched to 2.796×10^{-11} . **Always gradient-check**.
12. Fourteen lines of NumPy reach **0.9711** on load_digits; MLPClassifier gets 0.9756 and gradkit 0.9667 — a spread of four test rows.
13. All-zero initialisation gives **0.1000**, and all-0.1 gives 0.2356, because identical units stay identical forever. Use He for ReLU, Xavier for sigmoid.
14. An RNN shares one weight set across time; BPTT is backprop on the unrolled graph with gradients summed. $\partial h_T / \partial h_0$ is a product of T factors, so it vanishes or explodes: on a remember-the-first-bit task, accuracy 1.0000 at $T = 10$ and **0.4925** at $T = 40$.

Before the next lecture

Read

notes.pdf — the perceptron rule traced row by row, the backpropagation derivation with every partial derivative written out, and **practice questions with full worked solutions**, three of which are forward and backward passes by hand.

Do

Reproduce the 2–2–1 derivation with a pen and no notes: forward to $L = 0.1429003882$, then all nine gradients. Then change t from 1.0 to 0.0 and redo it. If you can do that, you can answer any backpropagation question that can be asked.

The habit to take away

Gradient-check every backward pass you write. Central difference, float64, $h = 10^{-6}$. It takes four lines and it saves the week you would otherwise spend blaming the learning rate.

And the habit to unlearn

Do not treat a network as a black box that “learns”. It multiplies, it activates, and it multiplies the derivatives back. Every one of those steps is arithmetic you can check by hand, and you should, once.

Materials: <https://github.com/tali7c/Applied-Machine-Learning> · Library to read: <https://github.com/tali7c/gradkit>

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 8–9.; James et al., *An Introduction to Statistical Learning with Python*, Ch. 8–9.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 4–5, 7, 14.; Zhang et al., *Dive into Deep Learning*, Ch. 5.